Plan and Manage an Azure AI Solution (25-30%)

- **Foundry structure:** A Foundry resource contains isolated projects. Projects contain models, agents, tools, knowledge, deployments, and connections.
- **Portal and SDK:** Use `ai.azure.com` and `AIProjectClient` to manage Foundry project assets.
- **Specialized tools:** Use Language, Speech, Translator, Document Intelligence, and Content Understanding for focused tasks instead of an LLM when they provide a more reliable fixed result.
- **Standard deployment:** Supports variable throughput without reserved capacity. Global Standard can route data outside a required region; Global Provisioned requires reserved throughput.
- **Deployment type:** Standard provides real-time REST access and key authentication without consuming virtual-machine vCPU quota. Batch is for offline work and self-hosted containers consume vCPU quota.
- **Model version policy:** Choose **Once the current version expires** when stable model behavior matters more than immediate upgrades.
- **Model selection:** Compare quality, attack success rate, throughput, cost, region support, modality, and fine-tuning support.
- **DefaultAzureCredential:** Uses Microsoft Entra managed identity in Azure and developer credentials locally for keyless authentication.
- **Least privilege:** Use **Cognitive Services OpenAI User** for inference and **Storage Blob Data Reader** to read blobs.
- **Key Vault access:** A system-assigned managed identity needs the **Key Vault Secrets User** role to read secrets.
- **Project connections:** Centralize credentials and configuration so many apps or agents can reuse a managed connection.
- **Key Vault connection:** Use category `AzureKeyVault` with `AccountManagedIdentity` authentication for a keyless connection.
- **Content Safety guardrails:** Configure checks for user input, output, tool responses, and tool calls; use **Block** when content must not continue.
- **Prompt Shields:** Detect indirect prompt injection in retrieved content, documents, and user input. Image moderation does not detect hidden instructions or steganography.
- **Spotlighting:** Makes external content less trusted than system instructions, reducing the effect of indirect injection.
- **Private networking:** Use a private endpoint plus VPN or ExpressRoute to securely analyze on-premises confidential data.
- **Keys:** Query keys are read-only; admin keys manage indexes and resources.
- **Key rotation:** Switch the app to the secondary key, regenerate the compromised primary key, then switch the app back to the primary key.
- **Model resources:** `OpenAI` creates Azure OpenAI resources; `CognitiveServices` creates a multi-service resource with one key and endpoint.
- **Customer-managed keys:** Create an `OpenAI` resource and set `--encryption` JSON with `keySource: Microsoft.KeyVault` to protect data with a customer-managed key.
- **DALL-E deployment:** Use Microsoft Foundry with Azure CLI; Microsoft Graph is not an Azure AI model-deployment tool.
- **Simple service access:** An endpoint URI and subscription key need less administration than OAuth, SAS, or certificate-based identity.
- **CI/CD evaluations:** GitHub Actions should use Azure Login with OIDC to avoid long-lived credentials. Set the evaluation failure action to **Fail** so unsuitable pull requests cannot merge.
- **GitHub Actions project target:** Set `project-endpoint` to the Foundry project endpoint; a tenant ID or model deployment name does not identify the project.
- **Telemetry privacy:** Set `enable_content_recording=False` to exclude prompts, completions, and tool payloads from telemetry attributes.
- **Tracing:** OpenTelemetry with Application Insights shows model calls, tools, tokens, timing, and nested parent-child spans for each run.
- **Application Insights:** Required for Foundry Control Plane views of errors, runs, token usage, and traces.
- **Diagnostics:** `RequestResponse` logs include request payloads, response information, status codes, and completions in Log Analytics.
- **Capacity signals:** Model Availability Rate shows model-side availability; Provisioned Utilization shows provisioned capacity pressure.
- **Cost signals:** Token usage metrics identify input, output, and tool cost drivers.
- **Rate limits:** Use exponential backoff with jitter for HTTP 429 responses; immediate retries usually fail again.

- **Temperature:** Lower values make output more consistent; higher values increase variation. It does not solve incomplete responses.
- **Adaptive reasoning:** `output_config.effort: "low"` reduces latency and internal token use for simple tasks.
- **Workflows:** A sequential workflow gives deterministic order. An **Ask a question** node creates a human approval gate.
- **Session memory:** Scope `session` keeps memory only during the current session and clears it when the session ends.
- **User isolation:** `{{$userId}}` resolves to the authenticated user identity for separate per-user memory.
- **Responsible AI:** Fairness, reliability and safety, privacy and security, inclusiveness, transparency, and accountability are the six principles.
- **Transparency:** Tell users how their data is processed and used.

Implement Generative AI and Agentic Solutions (30-35%)

- **Direct model calls:** Use an Azure OpenAI endpoint for OpenAI-compatible Chat Completions and Responses APIs. Use a Foundry project endpoint for project agents, connections, and tools.
- **Chat state:** Chat Completions requires the app to resend the full message history. Responses API stores state server-side and uses `previous_response_id`.
- **Few-shot prompting:** Provide a small set of examples, usually 1-10, to guide output without retraining.
- **Zero-shot prompting:** Gives instructions without examples. Chain of thought asks for reasoning steps; it is not example-based learning.
- **System instructions:** Define the agent's role and response boundaries.
- **LLMs:** Support detailed language generation and complex multi-step reasoning. SLMs can be less capable for deep reasoning.
- **Multimodal models:** Process text and images together.
- **Response completeness:** Increase `max_tokens` when output is cut short. Increasing temperature adds variety, not completeness.
- **Reflection and retry:** Check required fields or clauses, then regenerate when the response is incomplete.
- **Optimization order:** Improve system prompts first, use RAG for missing current or private facts, and fine-tune for a consistent style or output format.
- **Fine-tuning:** Does not add current factual knowledge; use RAG for facts that change.
- **Function tools:** The model requests a function call, but the host application executes the function and returns its result.
- **OpenAPI tools:** Define an API-key `securityScheme` in the OpenAPI specification. The project connection supplies the key value; per-operation headers are repetitive and a bearer scheme is the wrong authentication type.
- **Agent flow:** `create_agent()` -> `create_thread()` -> `create_message()` -> `create_and_process_run()` -> `list_messages()`.
- **Thread:** Persistent conversation history. A run executes an agent against a thread.
- **Conversation continuity:** Reusing a conversation ID restores messages, tool calls, and outputs across sessions.
- **Persistent memory:** Storage-backed agent memory can retain user preferences across separate sessions.
- **Conversation:** The durable **Conversation** component preserves messages, tool calls, and tool outputs across turns and sessions.
- **Get an agent:** Use `project_client.agents.get(agent_name=...)` to retrieve an existing agent configuration.
- **File search:** Grounds responses in uploaded documents and avoids sending oversized content directly in a request.
- **Code Interpreter:** Runs calculations and analysis in a sandbox.
- **Bing Grounding:** Retrieves current public web information.
- **`tool_choice="required"`:** Forces at least one tool call before the final answer. `tool_choice={"type": "bing_grounding"}` permits only Bing grounding.
- **MCP:** A standard protocol that connects agents to external services. The model can select discovered tools based on the task.
- **MCP tools:** Azure Language, Speech, and Document Intelligence can be exposed through MCP servers.
- **RAG:** Grounds model answers in retrieved authoritative content.
- **Agentic RAG:** Plans retrieval using conversation context, retrieves several chunks, and can run retrievals in parallel. Classic RAG is single-pass retrieval.
- **Groundedness:** Measures whether claims are supported by retrieved source content.
- **Relevance:** Measures whether the answer addresses the user query. Retrieval measures whether retrieved content matches the expected grounding context.
- **Evaluators:** Risk and Safety metrics assess harmful content; AI Quality metrics assess groundedness and relevance.
- **Protected material:** Is evaluated separately from harmful-content severity.
- **Power Fx conditions:** `Not(IsBlank(Local.Var01))` continues only when a local answer exists. `{Upper(Local.Var01)}` interpolates that value in uppercase.
- **Approval workflows:** `ask_question` pauses a YAML workflow for external approval; continue only when the approval value is `"approved"`.
- **Orchestration:** Use concurrent for independent parallel work, sequential for fixed pipelines, group chat for collaboration, handoff for specialist ownership, and Magentic for manager-led adaptive planning.

Implement Computer Vision Solutions (10-15%)

- **Multimodal messages:** Put text and images in the user message content array as `type: "text"` and `type: "image_url"` items.
- **Image input:** Use a remote URL or a Base64 data URL such as `...` for a local image.
- **Product fidelity:** Set `input_fidelity="high"` to preserve a reference product's recognizable visual characteristics.
- **Inpainting:** Use a mask to edit only selected areas and preserve the rest of an image. Image variation changes the complete image.
- **Image moderation:** Classifies uploaded images for harmful-content severity; do not rely on OCR keyword scanning for this job.
- **Vision detection:** Detects objects, brands, and logos and returns names, confidence scores, and bounding boxes.
- **Bounding boxes:** `X, Y` are the top-left corner and `W, H` are width and height. The bottom-right corner is `X+W, Y+H`.
- **Video generation:** `client.videos.create()` starts asynchronous generation; poll `client.videos.retrieve(video.id)` for status.
- **Custom Vision classifier:** Create a project, upload and tag images, then train the classifier.
- **Classification:** Requires class tags. Object detection requires a bounding box for each tagged object.
- **Precision:** $Precision = TP/(TP + FP)$. A precision of 100% means there are no false positives.
- **Recall:** $Recall = TP/(TP + FN)$. It measures how many actual positive items the model finds.
- **Accessibility:** Generate accurate captions and alt text that describe visual evidence.

Implement Text Analysis Solutions (10-15%)

- **Azure Language:** Provides language detection, sentiment, key phrases, summarization, named entity recognition, entity linking, and PII detection.
- **Sentiment:** Overall sentiment can be positive, negative, neutral, or mixed.
- **Custom NER:** Improve low precision by replacing broad labels with specific entity types such as `Phone` and `Email`; do not lower confidence thresholds.
- **PII redaction:** Replaces sensitive values with category labels such as `[PERSON]` and `[EMAIL]`; there is no `Contact` super-category.
- **Content Safety:** `AnalyzeTextOptions(text=comment)` with `client.analyze_text()` classifies harmful-content severity, not PII.
- **Text limits:** Azure Language supports up to 5,120 characters per document and 1,000 items per request.
- **Immersive Reader:** Provides an assistive reading experience for learners with dyslexia with minimal integration.
- **Real-time speech to text:** Processes streaming audio with about 100 ms latency for live conversations and turn-taking.
- **Batch transcription:** Processes recordings after they finish; it is unsuitable for live turn-taking.
- **Speech SDK:** Use `SpeechConfig -> AudioConfig -> SpeechRecognizer -> recognize_once_async()` for speech to text.
- **Text to speech:** Use `SpeechConfig -> AudioOutputConfig -> SpeechSynthesizer -> speak_text_async()`.
- **Custom Speech:** REST calls require the custom speech **Project ID**. An expired custom model silently falls back to the latest base model for the same locale.
- **SSML:** Use `<phoneme>` to control the pronunciation of technical terms without training a custom voice.
- **Voice Live:** Uses a `wss://` WebSocket for low-latency streaming. Stop playback when `input_audio_buffer.speech_started` signals barge-in.
- **Translator:** Supports text translation, document translation, transliteration, and more than 90 languages.
- **Mixed-language input:** Split text into single-language segments before translating for better accuracy.
- **Translation:** Converts meaning, for example Chinese text to English text. Transliteration converts a writing system into another script.
- **Speech translation:** Uses `SpeechTranslationConfig` to translate recognized speech into target languages.
- **Language resource networking:** A Language resource plus private endpoint plus VPN or ExpressRoute secures analysis of confidential on-premises documents.

Implement Information Extraction Solutions (10-15%)

- **Azure AI Search pipeline:** Data source -> Indexer -> Skillset -> Index -> Search.
- **Indexer:** Connects a data source to a skillset for extraction and enrichment, then stores data in the search index.
- **Index-time enrichment:** Skills run when the indexer runs. Adding a skill has no effect until the indexer is run again.
- **Text Split:** Creates searchable chunks. The Azure OpenAI Embedding skill generates vectors for those chunks.
- **Vector search:** Uses embeddings to match semantically similar wording.
- **Hybrid search:** Combines exact keyword/code matches with vector matches for natural-language descriptions.
- **Semantic ranking:** Re-ranks results by contextual relevance; it does not replace vector indexing.
- **Built-in skills:** OCR, NER, language detection, key phrases, sentiment, PII, translation, and image captions enrich indexed content.
- **Content Understanding skill:** Extracts text, images, tables, location metadata, and bounding polygons for a search skillset.
- **Knowledge Store:** Saves enriched content for downstream use and citations.
- **Object projection:** Stores complete hierarchical JSON. Table projection stores normalized rows for analysis in Azure Table Storage.
- **Document-level filtering:** Store allowed groups with documents, retrieve user group memberships, and filter queries with `groups/any(g: search.in(g, 'sales'))`.
- **Search connections:** Configure Azure AI Search at Foundry project level to let several client apps use the same managed connection.
- **Embedded PDF images:** Use an indexer to extract images into `normalized_images` for OCR-ready data while retaining source-document citation links.
- **Content Understanding standard mode:** Processes single, independent files at lower cost.
- **Content Understanding pro mode:** Processes multiple files and supports cross-document reasoning, such as validating an invoice against a purchase order and contract.
- **Custom analyzers:** Define field schemas with `extract`, `classify`, or `generate` methods. Route low-confidence fields, for example below 0.80, to human review.
- **Generated fields:** Use field type `string` with method `generate` for AI-generated descriptions.
- **Field evidence:** `estimateFieldSourceAndConfidence` returns per-field confidence scores and source locations.
- **Document Intelligence:** Use `prebuilt-read` for text and `prebuilt-layout` for tables and document structure.
- **Markdown output:** Set `output_content_format=ContentFormat.MARKDOWN` to retain tables and sections for RAG. Use `pages="1-3"` to analyze a PDF page range.
- **Document Search analyzer:** `prebuilt-documentSearch` creates RAG-optimized Markdown with semantic structure.